eth 및 snap 프로토콜 분석

개념적 배경

왜 별도의 애플리케이션 프로토콜이 필요한가?

devp2p는 안전한 전송 계층을 제공하지만, Ethereum 네트워크의 실제 기능(블록 동기화, 트랜잭션 전파, 상태 동기화)을 위해서는 애플리케이션 레벨의 프로토콜이 필요합니다.

eth 프로토콜의 역할:

- **블록체인 동기화**: 블록 헤더, 바디, 영수증 교환
- **트랜잭션 전파**: 새로운 트랜잭션의 네트워크 전파
- **상태 협상**: 네트워크 상태 및 포크 정보 교환

snap 프로토콜의 혁신:

- **빠른 동기화**: 상태 스냅샷 기반 동기화
- **병렬 처리**: 계정과 스토리지의 병렬 다운로드
- **효율성**: 기존 fast sync보다 훨씬 빠른 동기화

기존 방식의 한계와 개선

기존 fast sync의 문제점:

- **순차적 처리**: 헤더 → 상태 → 영수증 순서로 처리
- **단일 스레드**: 하나의 피어에서 순차적 다운로드
- **검증 오버헤드**: 모든 상태 변경을 재실행

snap sync의 혁신:

- **병렬 다운로드**: 여러 피어에서 동시 다운로드
- **스냅샷 기반**: 최종 상태만 다운로드
- **증명 기반 검증**: Merkle 증명으로 빠른 검증

eth 프로토콜 구현 분석

1. 프로토콜 버전 및 메시지 타입

```
// 지원되는 프로토콜 버전
const (
ETH68 = 68  // 기본 기능
ETH69 = 69  // 블록 범위 업데이트 추가
)

// 메시지 타입 정의
const (
StatusMsg = 0x00  // 상태 교환
NewBlockHashesMsg = 0x01  // 새 블록 해시 알림
TransactionsMsg = 0x02  // 트랜잭션 전파
GetBlockHeadersMsg = 0x03  // 블록 헤더 요청
BlockHeadersMsg = 0x04  // 블록 헤더 응답
GetBlockBodiesMsg = 0x05  // 블록 바디 요청
BlockBodiesMsg = 0x06  // 블록 바디 응답
NewBlockMsg = 0x07  // 새 블록 알림
```

```
NewPooledTransactionHashesMsg = 0x08  // 풀된 트랜잭션 해시
GetPooledTransactionsMsg = 0x09  // 풀된 트랜잭션 요청
PooledTransactionsMsg = 0x0a  // 풀된 트랜잭션 응답
GetReceiptsMsg = 0x0f  // 영수증 요청
ReceiptsMsg = 0x10  // 영수증 응답
BlockRangeUpdateMsg = 0x11  // 블록 범위 업데이트 (ETH69)
)
```

2. 상태 교환 메커니즘

ETH68 상태 패킷

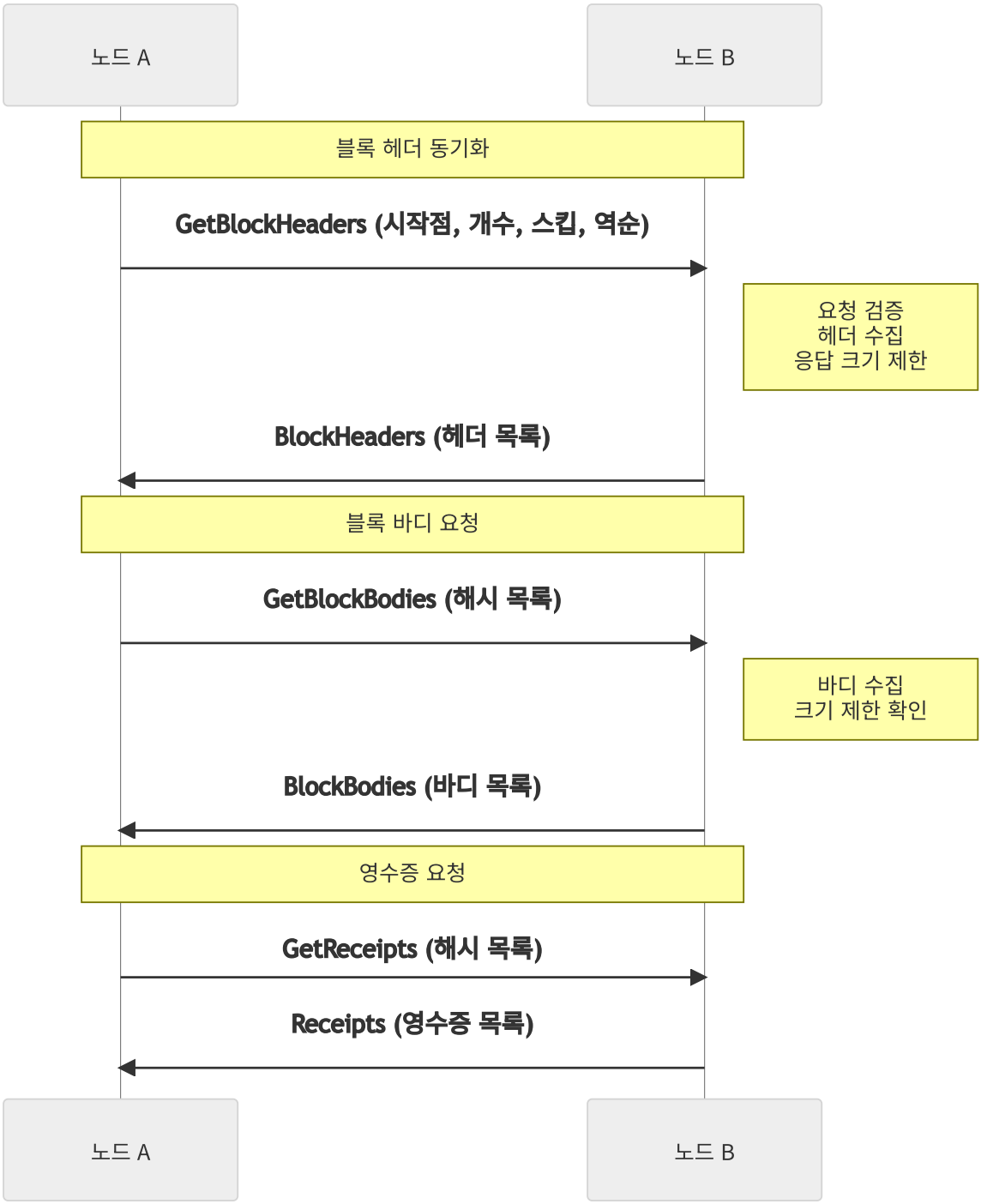
```
type StatusPacket68 struct {
ProtocolVersion  uint32  // 프로토콜 버전
NetworkID  uint64  // 네트워크 ID (1=메인넷)
TD *big.Int // 총 난이도
Head common.Hash // 현재 헤드 블록 해시
Genesis common.Hash // 제네시스 블록 해시
ForkID forkid.ID // 포크 식별자
}
```

ETH69 상태 패킷 (개선된 버전)

```
type StatusPacket69 struct {
```

```
ProtocolVersion uint32 // 프로토콜 버전

NetworkID uint64 // 네트워크 ID

Genesis common.Hash // 제네시스 블록 해시

ForkID forkid.ID // 포크 식별자

// 사용 가능한 블록 범위 정보

EarliestBlock uint64 // 가장 오래된 블록 번호

LatestBlock uint64 // 최신 블록 번호

LatestBlockHash common.Hash // 최신 블록 해시

}
```

3. 블록 헤더 교환 시스템



헤더 요청 구조

```
type GetBlockHeadersRequest struct {
Origin HashOrNumber // 시작점 (해시 또는 번호)
Amount  uint64  // 요청할 헤더 수
```

```
Skip uint64 // 건너뛸 블록 수

Reverse bool // 역순 여부

}


type HashOrNumber struct {

Hash common.Hash // 블록 해시로 지정

Number uint64 // 블록 번호로 지정

}
```

4. 트랜잭션 전파 메커니즘

```
graph TD

A[새 트랜잭션 생성] --> B{트랜잭션 검증}

B -->|유효| C[로컬 풀에 추가]

B -->|무효| D[거부]

C --> E[피어들에게 해시 전파]

E --> F[NewPooledTransactionHashes]

F --> G[관심있는 피어들이 요청]

G --> H[GetPooledTransactions]

H --> I[PooledTransactions 응답]

I --> J[피어의 풀에 추가]

J --> K[추가 전파]

style A fill:#e1f5fe
```

```
style C fill:#c8e6c9

style D fill:#ffcdd2
```

트랜잭션 전파 최적화

해시 기반 전파:

```go
type NewPooledTransactionHashesPacket []common.Hash


// 장점:
// - 네트워크 대역폭 절약
// - 중복 전송 방지
// - 선택적 다운로드 가능
```

풀 트랜잭션 요청:

```go
type GetPooledTransactionsPacket struct {
RequestId  uint64
Hashes []common.Hash
}


// 응답 크기 제한
const  softResponseLimit = 2 * 1024 * 1024
```

5. 블록 전파 시스템

새 블록 알림

```
type NewBlockHashesPacket []struct {
Hash common.Hash // 블록 해시
Number  uint64  // 블록 번호
}

type NewBlockMsg struct {
Block *types.Block // 전체 블록 데이터
TD *big.Int // 총 난이도
}
```

전파 전략

```
graph  LR
A[새 블록 생성]  -->  B{블록 크기 확인}
B -->|작음|  C[NewBlock 직접 전파]
B -->|큼|  D[NewBlockHashes 전파]
C -->  E[즉시 블록 데이터 전송]
D -->  F[해시만 전송]
F -->  G[피어가 필요시 요청]
G -->  H[GetBlockBodies/Headers]
style C fill:#c8e6c9
```

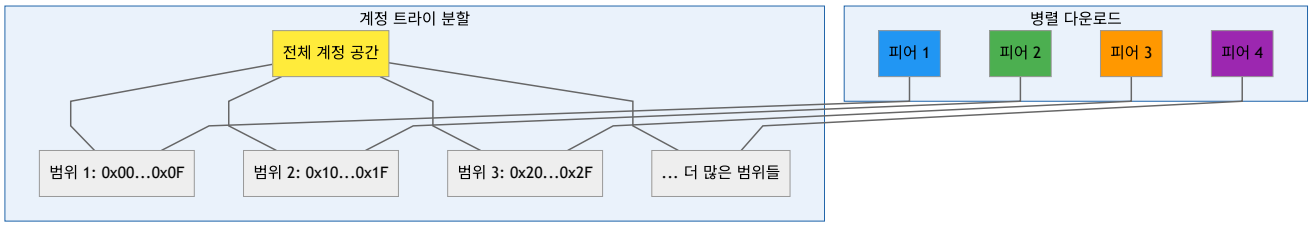
```
style D fill:#fff3e0
```

snap 프로토콜 구현 분석

1. 프로토콜 구조

```
// snap 프로토콜 메시지 타입
const (
GetAccountRangeMsg = 0x00  // 계정 범위 요청
AccountRangeMsg = 0x01  // 계정 범위 응답
GetStorageRangesMsg = 0x02  // 스토리지 범위 요청
StorageRangesMsg = 0x03  // 스토리지 범위 응답
GetByteCodesMsg = 0x04  // 바이트코드 요청
ByteCodesMsg = 0x05  // 바이트코드 응답
GetTrieNodesMsg = 0x06  // 트라이 노드 요청
TrieNodesMsg = 0x07  // 트라이 노드 응답
)
```

2. 계정 범위 동기화



계정 범위 요청

```
type GetAccountRangePacket struct {
ID  uint64  // 요청 ID
Root common.Hash // 계정 트라이 루트
Origin common.Hash // 시작 계정 해시
Limit common.Hash // 끝 계정 해시
Bytes  uint64  // 응답 크기 제한
}
```

계정 범위 응답

```
type AccountRangePacket struct {
ID  uint64  // 요청 ID
Accounts []*AccountData // 계정 데이터 목록
Proof [][]byte  // Merkle 증명
}
```

```go
type AccountData struct {
Hash common.Hash // 계정 해시
Body rlp.RawValue // 슬림 형식의 계정 데이터
}
```

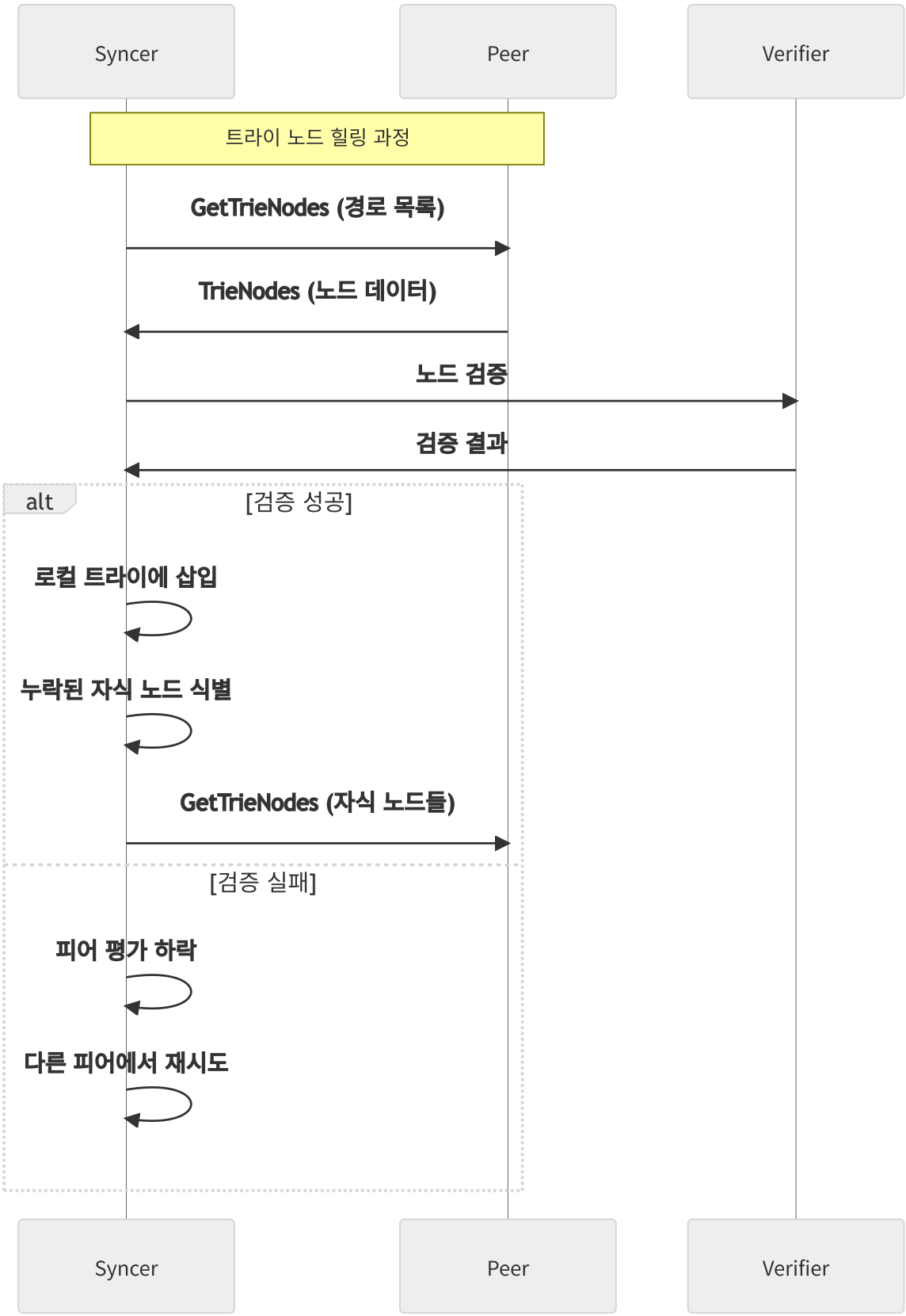
3. 스토리지 범위 동기화

```go
type GetStorageRangesPacket struct {
ID  uint64  // 요청 ID
Root common.Hash // 상태 루트
Accounts []common.Hash // 대상 계정들
Origin []byte  // 시작 스토리지 키
Limit []byte  // 끝 스토리지 키
Bytes  uint64  // 응답 크기 제한
}

type StorageRangesPacket struct {
ID  uint64  // 요청 ID
Slots [][]*StorageData // 계정별 스토리지 슬롯들
Proof [][]byte  // Merkle 증명
}
```

4. 바이트코드 동기화

```go
type GetByteCodesPacket struct {
ID  uint64  // 요청 ID
Hashes []common.Hash // 바이트코드 해시들
Bytes  uint64  // 응답 크기 제한
}


type ByteCodesPacket struct {
ID  uint64  // 요청 ID
Codes [][]byte  // 바이트코드들 (nil = 누락)
}
```

5. 트라이 노드 힐링


Syncer
Peer
Verifier
트라이 노드 힐링 과정
GetTrieNodes (경로 목록)
TrieNodes (노드 데이터)
노드 검증
검증 결과
alt
[검증 성공]
로컬 트라이에 삽입
누락된 자식 노드 식별
GetTrieNodes (자식 노드들)
[검증 실패]
피어 평가 하락
다른 피어에서 재시도
Syncer
Peer
Verifier

성능 최적화 기법

1. 응답 크기 제한

```
const (
softResponseLimit = 2 * 1024 * 1024  // 2MB 소프트 제한
maxHeadersServe = 1024  // 최대 헤더 수
maxBodiesServe = 1024  // 최대 바디 수
maxReceiptsServe = 1024  // 최대 영수증 수
)

// 동적 크기 조정
func  serviceGetBlockHeaders(chain *core.BlockChain, query *GetBlockHeadersReque
var (
bytes common.StorageSize
headers []rlp.RawValue
)
for  len(headers) < int(query.Amount) && bytes < softResponseLimit {
// 헤더 수집 및 크기 확인
if  header := getNextHeader();  header != nil {
if  rlpData, err := rlp.EncodeToBytes(header);  err == nil {
headers = append(headers, rlp.RawValue(rlpData))
bytes += common.StorageSize(len(rlpData))
}
}
```

```
}

return headers

}
```

2. snap 동기화 최적화

동시성 제어

```
const (

accountConcurrency = 16  // 계정 동시 다운로드 수

storageConcurrency = 16  // 스토리지 동시 다운로드 수

minRequestSize = 64 * 1024  // 최소 요청 크기

maxRequestSize = 512 * 1024  // 최대 요청 크기

)
```

적응적 요청 크기

```
type accountRequest struct {

peer  string

origin common.Hash

limit common.Hash

// 성능 추적

time time.Time

timeout *time.Timer
```

```
}


// 피어 성능에 따른 요청 크기 조정
func (s *Syncer) adjustRequestSize(peer *Peer, responseTime time.Duration) {
if  responseTime < targetResponseTime {
// 빠른 응답 → 요청 크기 증가
peer.requestSize = min(peer.requestSize*1.1, maxRequestSize)
} else {
// 느린 응답 → 요청 크기 감소
peer.requestSize = max(peer.requestSize*0.9, minRequestSize)
}
}
```

3. 메모리 관리

```
type readBuffer struct {
data []byte
pos  int
}


func (r *readBuffer) reset() {
r.pos = 0
// 버퍼 재사용으로 GC 압박 감소
```

```
}


// 슬라이스 확장 최적화
func growslice(slice []byte, size int) []byte {
if cap(slice) >= size {
return slice[:size]
}
// 2배씩 확장하여 재할당 최소화
newCap := cap(slice)
if newCap == 0 {
newCap = size
}
for newCap < size {
newCap *= 2
}
newSlice := make([]byte, size, newCap)
copy(newSlice, slice)
return newSlice
}
```

실제 사용 예시

1. eth 프로토콜 핸들러 구현

```go
func makeEthProtocol(backend Backend) p2p.Protocol {
return p2p.Protocol{
Name: "eth",
Version: ETH69,
Length: 18,
Run: func(p *p2p.Peer, rw p2p.MsgReadWriter) error {
peer := NewPeer(ETH69, p, rw, backend.TxPool())
defer peer.Close()
return backend.RunPeer(peer, func(peer *Peer) error {
return handleEthProtocol(backend, peer)
})
},
}
}


func handleEthProtocol(backend Backend, peer *Peer) error {
for {
if err := handleMessage(backend, peer); err != nil {
return err
}
}
}
```

2. 블록 헤더 요청 처리

```
func handleGetBlockHeaders(backend Backend, msg Decoder, peer *Peer) error {
var query GetBlockHeadersPacket
if err := msg.Decode(&query); err != nil {
return err
}
// 헤더 수집
headers := ServiceGetBlockHeadersQuery(
backend.Chain(),
query.GetBlockHeadersRequest,
peer,
)
// 응답 전송
return peer.ReplyBlockHeadersRLP(query.RequestId, headers)
}
```

3. snap 동기화 시작

```
func startSnapSync(backend Backend, peers []*Peer) error {
syncer := &Syncer{
backend: backend,
peers: peers,
```

```
tasks: make(map[common.Hash]*accountTask),

}

// 계정 범위 작업 생성

root := backend.Chain().CurrentBlock().Root()

for  i := 0;  i < accountConcurrency;  i++ {

task := &accountTask{

root: root,

origin: calculateRangeStart(i, accountConcurrency),

limit: calculateRangeEnd(i, accountConcurrency),

}

syncer.tasks[task.origin] = task

}

return  syncer.run()

}
```

4. 트랜잭션 전파

```
func (p *Peer) SendNewTransactionHashes(hashes []common.Hash) error {

// 해시만 전송하여 대역폭 절약

return  p2p.Send(p.rw, NewPooledTransactionHashesMsg, hashes)

}


func (p *Peer) RequestPooledTransactions(hashes []common.Hash) error {
```

```
id := rand.Uint64()

packet := GetPooledTransactionsPacket{

RequestId: id,

Hashes: hashes,

}

return p2p.Send(p.rw, GetPooledTransactionsMsg, packet)

}
```

에러 처리 및 복구

1. 네트워크 에러 처리

```
func handleNetworkError(err error, peer *Peer) error {

switch {

case errors.Is(err, io.EOF):

// 연결 종료

return err

case errors.Is(err, errMsgTooLarge):

// 메시지 크기 초과

peer.Log().Warn("Message too large", "err", err)

return err

case errors.Is(err, errInvalidMsgCode):

// 잘못된 메시지 코드
```

```
return DiscProtocolError
default:
return err
}
}
```

2. 동기화 실패 복구

```
func (s *Syncer) handleSyncFailure(task *accountTask, err error) {
// 실패한 작업을 다른 피어에게 재할당
if peer := s.selectBestPeer(); peer != nil {
s.assignTask(task, peer)
} else {
// 사용 가능한 피어가 없으면 대기
s.pendingTasks = append(s.pendingTasks, task)
}
}
```

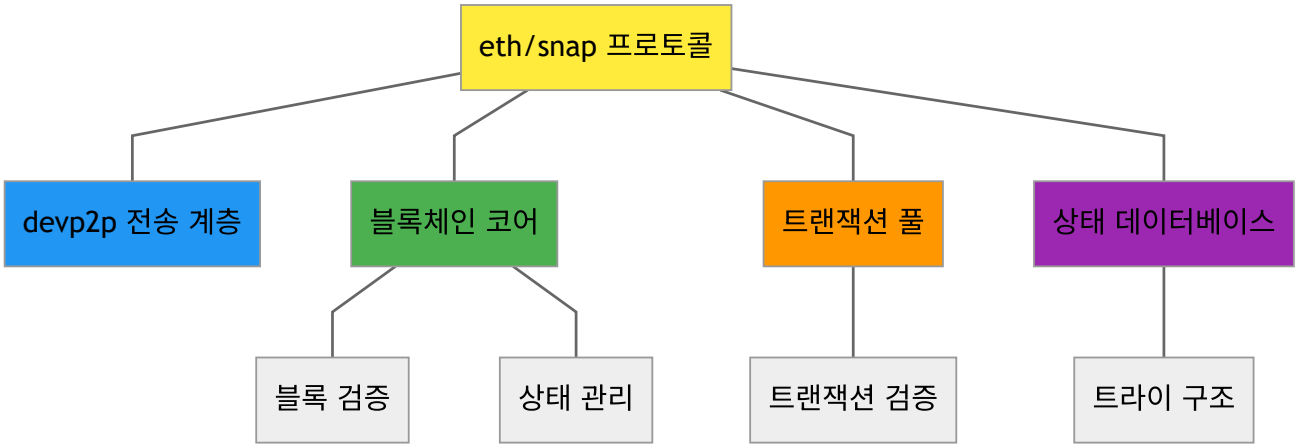
3. 피어 품질 관리

```
type PeerQuality struct {
responseTime time.Duration
successRate float64
```

```
bandwidth  uint64

}



func (p *Peer) updateQuality(responseTime time.Duration, success  bool) {

p.quality.responseTime = (p.quality.responseTime + responseTime) / 2

if  success {

p.quality.successRate = p.quality.successRate*0.9 + 0.1

} else {

p.quality.successRate = p.quality.successRate * 0.9

}

}
```

관련 시스템

의존성 관계



상호작용 패턴

1. **eth → 블록체인**: 수신된 블록을 체인에 삽입
2. **snap → 상태DB**: 동기화된 상태를 데이터베이스에 저장
3. **트랜잭션 풀 → eth**: 새 트랜잭션을 네트워크에 전파
4. **블록체인 → eth**: 새 블록을 피어들에게 알림

영향 범위

직접 영향:

- 블록체인 동기화 속도
- 트랜잭션 전파 효율성
- 네트워크 대역폭 사용량

간접 영향:

- 노드 시작 시간
- 네트워크 전체 성능
- 사용자 경험

문제 해결 가이드

일반적인 문제들

1. **동기화 속도 저하**

```
증상: 블록 동기화가 매우 느림

원인:

- 피어 품질 저하

- 네트워크 대역폭 제한
```

- 디스크 I/O 병목

해결방법:

- 더 많은 피어 연결

- 네트워크 설정 최적화

- SSD 사용 권장

2. snap 동기화 실패

증상: snap sync가 중간에 멈춤

원인:

- 피어의 스냅샷 불일치

- 네트워크 불안정

- 메모리 부족

해결방법:

- 다른 피어로 재시도

- 메모리 증설

- fast sync로 대체

3. 트랜잭션 전파 지연

증상: 트랜잭션이 네트워크에 늦게 전파됨

원인:

- 피어 연결 수 부족

- 가스 가격 너무 낮음

- 네트워크 혼잡

```
해결방법:

- 더 많은 피어 연결

- 적절한 가스 가격 설정

- 트랜잭션 풀 설정 조정
```

성능 튜닝

```
// eth 프로토콜 최적화 설정

config := eth.Config{

SoftResponseLimit: 4 * 1024 * 1024, // 4MB로 증가

MaxHeadersServe: 2048, // 더 많은 헤더 제공

MaxBodiesServe: 2048, // 더 많은 바디 제공

}


// snap 동기화 최적화

snapConfig := snap.Config{

AccountConcurrency: 32, // 동시성 증가

StorageConcurrency: 32, // 동시성 증가

MaxRequestSize: 1024 * 1024, // 1MB로 증가

}
```

모니터링 메트릭

- `eth_protocol_messages_total` : 프로토콜 메시지 수
- `snap_sync_progress` : snap 동기화 진행률
- `peer_response_time` : 피어 응답 시간
- `block_propagation_time` : 블록 전파 시간

이 분석을 통해 geth의 eth와 snap 프로토콜이 어떻게 효율적인 블록체인 동기화와 트랜잭션 전파를 구현하는지, 그리고 snap 프로토콜이 어떻게 혁신적인 상태 동기화 최적화를 제공하는지 완전히 이해할 수 있습니다.